Техническое задание

Мульти-Таймфреймный дашборд EURUSD с онлайн мета-моделью (MT5 + LightGBM + SGD)

Версия: 2.0
Файл: `dashboard.py`
Дата: 2025

---

1. Назначение

Интерактивный веб-дашборд для:

- Визуализации и пошагового тестирования LightGBM-моделей на 5 таймфреймах (5m, 10m, 15m, 30m, 60m) по паре EURUSD
- Онлайн-обучения мета-модели, комбинирующей прогнозы всех TF в единый прогностический сигнал
- Оценки качества прогнозов в реальном времени (MAE, Direction Accuracy, корреляция)

---

2. Источник данных

- MT5: `copy_rates_from_pos("EURUSD", TIMEFRAME_M5, 0, 15000)`
- Фильтр: только будни (`dayofweek < 5`)
- Кэширование: `@st.cache_data(ttl=300)`
- Ресэмплинг M5 → 10m, 15m, 30m, 60m (OHLC: `first / max / min / last`, volume: `sum`)

---

3. Целевая переменная

- Гауссова конволюция: `scipy.ndimage.gaussian_filter1d(close, sigma)` (настраиваемо в UI, default = 5)
- `delta_gauss = gauss.diff()`
- `target = delta_gauss.shift(-1)` — изменение сглаженной цены на 1 бар вперёд
- Модели предсказывают `delta_gauss`, а не уровень цены

---

4. Признаки базовых моделей

Для каждого TF формируется одинаковый набор из 50 лаговых признаков:

Группа	Признаки	Формула
Гаусс лаги	`gc_lag_1..10`	`delta_gauss.shift(i)`

Open diff	`open_dif_1..10`	`open.diff().shift(i)`
High diff	`high_dif_1..10`	`high.diff().shift(i)`
Low diff	`low_dif_1..10`	`low.diff().shift(i)`
Close diff	`close_dif_1..10`	`close.diff().shift(i)`

Все признаки используют `.shift(1)` — гарантия отсутствия утечек данных.

Дополнительно для визуализации: `ma20 = close.rolling(20).mean()`.

---

5. Базовые модели (уровень 1)

- Алгоритм: `LightGBM Regressor`
- Параметры: `n_estimators=200`, `max_depth=5`, `learning_rate=0.05`, `verbose=-1`
- Отдельная модель на каждом TF (5 моделей)
- Полное переобучение на последних N барах при каждом шаге симуляции:

TF	Диапазон	Default
5m	100–1000	300
10m–60m	50–500	150

---

6. Мета-модель (уровень 2, online learning)

Назначение

Комбинирует 5 сигналов базовых моделей в один прогностический признак с самообучением на потоке данных.

Алгоритм

```
Python


SGDRegressor(
    loss="squared_error",
    learning_rate="constant",
    eta0=<slider>,          # 0.001..0.1, default 0.01
    alpha=<slider>,         # 1e-5..1e-2, default 1e-4
    penalty="l2",
    warm_start=True,
    max_iter=1, tol=None,
)
```

Входные признаки (5 штук)

Предсказания базовых моделей, нормализованные bar-normalized:

```
text
```

```
feat_i = pred_i * 10000 / sqrt(TF_i_minutes / 5)
```

Нормализация приводит все прогнозы к масштабу «пипсы на 5-минутный интервал».

Логика онлайн-обучения (методология prequential)

1. На баре `k` формируется вектор фичей из 5 базовых предсказаний
2. Мета-модель делает `meta_pred` (или среднее — на холодном старте, до первого `partial_fit`)
3. Пара `(X_k, step_time_k)` кладётся в буфер ожидания факта
4. На баре `k+1` появляется истинный `delta_gauss[k+1]` — это факт для шага `k`
5. Вызывается `partial_fit(X_k, y_true)` → модель адаптируется
6. В истории обновляется `y_true` для метрик

Свойства

- ✅ Истинный online learning (без переобучения на всей истории)
- ✅ Адаптация к дрейфу рынка (последние примеры влияют сильнее)
- ✅ Инспектируемость — можно смотреть веса `coef_` (какому TF модель доверяет)
- ✅ Быстро — не тормозит UI

---

7. Live-симуляция (пошаговая)

- Cut: последние 100 баров M5 откладываются в тестовый пул
- Кнопки:

- `Следующий бар` ➡️ — добавить 1 бар
- `+10` ⏩ — добавить 10 баров
- `Сброс` 🔁 — обнулить симуляцию, историю прогнозов и мета-модель

На каждом шаге:

1. Обновляется `current_m5 = train_pool + test_pool[:step]`
2. Пересчитывается гаусс и все признаки на всём окне
3. Для каждого TF:
    - Ресэмплинг + фичи
    - Находится последний завершённый бар `≤ step_time`
    - Обучается свежая LightGBM → предсказание
4. Мета-модель:
    - Дообучается на факте предыдущего шага (если появился)
    - Делает новый прогноз для текущего шага

Устойчивость

Реализован catch-up loop: если пользователь нажал `+10`, все промежуточные шаги отрабатываются последовательно, метамодель обучается на каждом.

---

8. Визуализация

Один синхронизированный график (2 подграфика, `shared_xaxes=True`)

#	Высота	Содержимое

1 (верх)	65%	Свечи EURUSD M5 (opacity 0.7) + MA20 (оранжевая) + вертикальная жёлтая пунктирная линия границы train/test
2 (низ)	35%	Прогнозы 5 базовых моделей + мета-сигнал (золотой ромб) + опционально факт `Δgauss 5m` (белый пунктир) + нулевая линия

Стилистика прогнозов на нижней панели

- Step-line (`line_shape="hv"`) — прогноз «действует» до следующего обновления
- Цвета TF: 5m🔵 / 10m🔴 / 15m🟢 / 30m🟣 / 60m🟤
- Мета: `#FFD700`, толщина 2.5, маркер diamond
- `skip_first` (0–10) — прячет начальные выбросы

Оси

- Rangebreaks: скрыты выходные (sat–mon)
- Тёмная тема `#1e1e1e`
- Rangeslider выключен

---

9. Sidebar — параметры UI

Параметр	Диапазон	Default

Train баров (5m)	100–1000	300
Train баров (10m–60m)	50–500	150
Sigma (Gauss)	1–20	5
Нормализация	`Bar-normalized / Пипсы / Raw`	Bar-normalized
Показать факт Δgauss 5m	bool	True
Показать мета-сигнал	bool	True
Skip first N точек	0–10	1
Meta learning rate	0.001–0.1	0.01

Meta L2 alpha	1e-5–1e-2	1e-4

---

10. Метрики мета-модели

Отображаются под графиком (только когда есть ≥5 обучающих примеров):

Метрика	Формула
Обучающих примеров	`count(y_true is not None)`
MAE (pips)	`mean(
Direction Accuracy	`mean(sign(y_true) == sign(meta_pred))`
Corr(pred, true)	`Pearson corr`

Дополнительно, в раскрывающемся блоке — веса мета-модели (`coef_` + `intercept_`), показывающие, какому TF модель доверяет больше.

11. Таблица текущих сигналов

DataFrame под метриками:

TF	Time	Δgauss (raw)	Norm	Dir
5m … 60m	время последнего бара TF	сырое значение	нормализованное	🟢 UP / 🔴 DOWN
🧠 META	время шага	—	значение мета-сигнала	🟢/🔴

12. Session state (структура)

Python

```
st.session_state = {
    "step": int,                          # текущий шаг симуляции
    "predictions": {tf: [ {time, step_time, delta_gauss, step}, ... ]},
    "meta_history": [ {time, step, meta_pred, y_true, components}, ... ],
    "meta_model": SGDRegressor,           # инкрементально обучаемая модель
    "meta_fitted": bool,                  # флаг первого partial_fit
    "meta_buffer": [ {features, step_time, step}, ... ],  # ожидание фактов
}
```

13. Технологический стек

Пакет	Версия	Назначение
`MetaTrader5`	≥ 5.0.45	Данные
`pandas`	≥ 2.0	Обработка
`numpy`	≥ 1.24	Вычисления
`lightgbm`	≥ 4.0	Базовые модели
`scikit-learn`	≥ 1.3	`SGDRegressor` (мета-модель)
`scipy`	≥ 1.11	`gaussian_filter1d`
`plotly`	≥ 5.18	Интерактивные графики

`streamlit`	≥ 1.30	Веб-интерфейс

---

14. Запуск

```Bash
pip install -r requirements.txt
streamlit run dashboard.py
```

Требования:

- Установленный терминал MetaTrader 5
- Активная авторизация MT5, доступ к символу `EURUSD` на таймфрейме M5
- Python 3.9+

---

15. Проверенная функциональность

- ✅ Загрузка и ресэмплинг данных MT5
- ✅ Без утечек данных (`.shift(1)` на всех признаках)
- ✅ Пошаговая симуляция с полным пересчётом на каждом шаге
- ✅ Онлайн-обучение мета-модели (prequential validation)
- ✅ Синхронизированный зум обоих подграфиков
- ✅ Скрытие выходных на оси времени
- ✅ Три режима нормализации для сравнения TF в едином масштабе
- ✅ Метрики качества мета-модели в реальном времени
- ✅ Инспекция весов мета-модели

---

16. Известные ограничения

1. Полное переобучение LightGBM на каждом шаге — на слабых машинах медленно; можно оптимизировать через `warm_start` или incremental learning
2. Мета-модель линейна — не улавливает нелинейных взаимодействий между TF; кандидаты на замену: `PassiveAggressiveRegressor`, `river.linear_model.PARegressor`, скользящий LightGBM
3. Cold start мета-модели — первые ~5–10 прогнозов — простое среднее фичей
4. Одна валютная пара — EURUSD зашита в код

---

17. Roadmap (потенциальные расширения)

- Выбор валютной пары и таймфреймов из UI
- Использование библиотеки `river` для полноценного online-ML с drift detection
- Расширение фичей меты: `abs(pred_i)`, `pred_i - pred_j`, `std(preds)`, время суток, волатильность
- Ensemble двух мета-моделей (быстрая линейная + медленная нелинейная)
- Экспорт истории прогнозов и метрик в CSV
- Бэктест торговой стратегии на основе мета-сигнала (PnL, Sharpe, DD)
- Сохранение/загрузка обученной мета-модели (`joblib`)